```
<p>Day Rate: @Model.DayRate.ToString("c")</p>
<p>Max Guests: @Model.MaxNumberOfGuests</p>
<p>Smoking permitted? @(Model.SmokingPermitted ? "Yes" : "No")</p>
```

Next you need a page handler that uses this partial to generate HTML. Add the following handler method to the City.cshtml.cs file after the existing `OnGetAsync` method.

Listing 11.3 The named handler that generated HTML as a `PartialViewResult`

```
public async Task<PartialViewResult> OnGetPropertyDetails(int id)
{
    var property = await _propertyService.FindAsync(id);
    return Partial("_PropertyDetailsPartial", property);
}
```

This is a named handler method. It takes an `int` as a parameter, representing the identity of the chosen property. It uses the property service to obtain the property details from the database and then passes it to the partial file, returning the rendered result. You need to inject the property service into the constructor of the `PageModel`.

Listing 11.4 Injecting the property service into the `CityModel` constructor

```
private readonly ICityService _cityService;
private readonly IPropertyService _propertyService;

public CityModel(ICityService cityService, IPropertyService propertyService)
{
    _cityService = cityService;
    _propertyService = propertyService;
}
```

The final steps involve some changes to the City.cshtml file. Iterate a collection of properties within the selected city, and render their details in this file. The name of the property is presented in an `h5` heading, which has